

Deep Dive: The Slice Selection Policy (SSP)

1 What is the Slice Selection Policy?

The Slice Selection Policy (SSP) is the decision-making core of the Active Spiking Perception (ASP) framework[cite: 7, 9]. It replaces the traditional fixed-order processing of 3D point cloud slices with a learned, dynamic sequence[cite: 8, 9].

Physically, the SSP is a lightweight cross-attention module containing approximately 2,000 parameters, which adds less than 0.2% to the overall model size[cite: 9, 37]. At each timestep, it reads the SNN’s current Leaky Integrate-and-Fire (LIF) membrane state and the precomputed geometry descriptors of unvisited anchor points to output a ranked priority for the remaining regions[cite: 9].

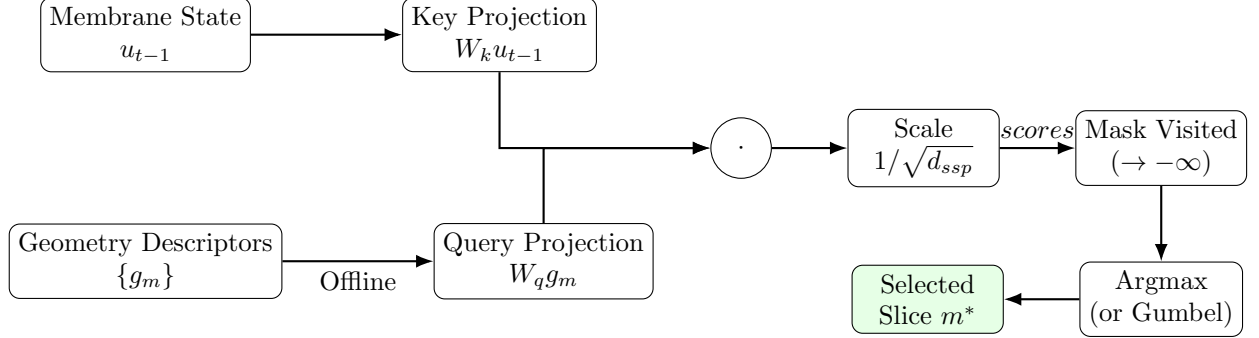
2 The Inputs

The SSP makes its decisions by looking at two distinct inputs at timestep t [cite: 90]:

- **The Membrane State** ($u_{t-1} \in \mathbb{R}^D$): This acts as the SNN’s “belief state”—a compressed mathematical summary of everything it has observed up to this point[cite: 33].
- **The Geometry Descriptors** ($g_m \in \mathbb{R}^6$): For each of the M Farthest Point Sampling (FPS) anchors, the system precomputes a 6-dimensional descriptor [cite: 80]. This vector includes the region’s centroid XYZ, its distance to the point cloud centroid, its intra-cluster spread, and its normalized point count[cite: 81, 82, 83, 84, 85]. These are computed offline before neural processing begins, requiring no MAC operations[cite: 87].

3 The SSP Internal Mechanism

The SSP uses a scaled dot-product attention mechanism to rank the unvisited slices [cite: 119].



3.1 Mathematical Formulation

The membrane state is projected into a “key” vector, and each geometry descriptor is projected into a “query” vector[cite: 118]. The priority score for anchor m is calculated as:

$$score(m) = \frac{(W_k u_{t-1})^\top (W_q g_m)}{\sqrt{d_{ssp}}} \quad (1)$$

where $W_k \in \mathbb{R}^{d_{ssp} \times D}$ and $W_q \in \mathbb{R}^{d_{ssp} \times 6}$ are trainable weight matrices, and $d_{ssp} = 64$ [cite: 119, 120]. To prevent the SNN from processing the same region twice, the scores of already visited anchors are masked to $-\infty$ [cite: 119].

4 Real-World Example: Classifying Objects

To understand why this is so efficient, consider how the SSP adapts to different physical objects.

For a chair, seeing the backrest first is highly discriminative [cite: 31]. Conversely, for a lamp, the base structure is the most informative region[cite: 31]. By allowing the SNN’s membrane potential to actively drive the observation order, the model zeroes in on these critical features immediately[cite: 33].

Wasting timesteps on regions the model is already confident about is energetically wasteful and computationally redundant[cite: 32]. By utilizing the SSP, the SNN successfully exits after inspecting only 2.4 out of 16 slices on average (at $\theta = 0.7$)[cite: 14].

5 Neuromorphic Compatibility

A key feature of the SSP is its extreme energy efficiency on neuromorphic hardware like Intel Loihi 2[cite: 15].

- The key projection matrix W_k is applied to the binary spike output of the previous temporal head step, meaning it requires only cheap Accumulate (AC) operations[cite: 122].
- The query projection $W_q g_m$ is precomputed offline once per input[cite: 123].
- The per-step computational cost is $\mathcal{O}(M \cdot d_{ssp})$ AC operations, which is negligible compared to running the backbone SNN[cite: 123].

6 Differentiability and Inference

Because selecting a single slice involves a discrete, non-differentiable choice, the SSP uses two different methods depending on the phase:

- **Training Phase:** The network precomputes backbone features for all slices in parallel[cite: 126]. The SSP applies a Gumbel-softmax function (with `hard=True`) to act as a straight-through estimator[cite: 127]. This creates an approximately one-hot vector that selects the feature differentially via a weighted sum, allowing the entire system to be trained end-to-end[cite: 127, 128].
- **Inference Phase:** The SNN drops the Gumbel-softmax and simply uses a hard argmax to pick the highest-scoring slice, running at $\mathcal{O}(1)$ overhead per step[cite: 11].